

SCREENFLIX

Apostila do Workshop

Do container ao código, do código à IA — um projeto como fio condutor.

DIA 1

Infra & Containers

DIA 2

**Programação
Pragmática**

DIA 3

IA no Desenvolvimento

Material de apoio para os participantes · 3 encontros de 1h30

Projeto de referência: **ScreenFlix** — FastAPI · PostgreSQL · Docker · Vue 3

Sumário

0 Introdução e preparação do ambiente

1 Dia 1 — Infra & Containers

[Por que containers](#) · [Dockerfile multi-stage](#) · [docker-compose](#) · [Comandos](#) · [Makefile](#)

2 Dia 2 — Programação Pragmática

[Ambientes modulares](#) · [Clean Architecture](#) · [Regra de dependência](#) · [Padrões](#) · [Trade-offs](#)

3 Dia 3 — IA no Desenvolvimento

[Claude Code](#) · [CLAUDE.md](#) · [Skills](#) · [Subagents](#) · [Agent Teams](#)

A Apêndices — Cheat sheet e glossário

INTRODUÇÃO

Bem-vindo ao Workshop ScreenFlix

Este workshop usa um projeto real — o **ScreenFlix**, um catálogo de filmes e séries — como estudo de caso vivo. Em vez de exemplos artificiais, você vai ver como as decisões de **infraestrutura**, **arquitetura** e **uso de IA** se conectam num único código que evolui ao longo de três encontros.

O que é o ScreenFlix?

Uma aplicação backend em **Python 3.13 + FastAPI** (um "monólito modular em camadas") com um frontend em **Vue 3 + Pinia**. Ela busca dados de filmes/séries numa API externa (OMDb), normaliza com um modelo de linguagem (OpenAI) e serve um catálogo via API REST.

Camada	Tecnologias
Backend	FastAPI, SQLAlchemy async, Pydantic, Structlog, uvicorn
Dados	PostgreSQL 18 (tabelas <code>media</code> e <code>episode</code>)
Infra	Docker (multi-stage), docker-compose, uv
Frontend	Vue 3, Vue Router, Pinia, Vite
Integrações	OMDb (ingestão) + OpenAI (normalização por schema)

Como usar esta apostila

Cada dia tem a mesma estrutura: **conceitos** → **o código do ScreenFlix** → **exercícios práticos** → **resumo**. Os blocos coloridos ajudam a navegar:

DEMONSTRAÇÃO

algo que será mostrado ao vivo — acompanhe no seu terminal.

MÃOS À OBRA

um exercício para você executar.

ATENÇÃO

um ponto de armadilha ou decisão importante de projeto.

Preparação do ambiente (faça antes do Dia 1)

```
# 1. Clonar o repositório
git clone <url-do-repo> screenflix && cd screenflix

# 2. Instalar as ferramentas
#   - Docker + Docker Compose      (Dia 1)
#   - uv → https://docs.astral.sh/uv/ (Dias 1 e 2)
#   - Node.js 20+                   (Dia 2, frontend)
#   - Claude Code CLI               (Dia 3)

# 3. Conferir versões
docker --version && docker compose version
uv --version
node --version
```

ATENÇÃO

Alguns itens (CI, migrations Alembic, persistência do banco no compose) **ainda não existem** no repositório — de propósito. Eles são construídos/estendidos **ao vivo** como exercícios (o **Makefile** já existe — nós o lemos, usamos e estendemos). A apostila sempre sinaliza o que já existe versus o que criamos.

Infra & Containers

Objetivo: empacotar a aplicação de forma reprodutível. Ao final você saberá ler/escrever um Dockerfile multi-stage, orquestrar serviços com docker-compose, usar os comandos do dia a dia e padronizar tudo num Makefile.

1.1 Por que containers?

O ScreenFlix depende de Python 3.13, PostgreSQL 18, bibliotecas de sistema (`libpq`) e variáveis de ambiente. Sem containers, cada pessoa reproduz isso na mão — a origem do clássico *"na minha máquina funciona"*. Containers entregam:

- **Reprodutibilidade** — a mesma imagem roda igual em qualquer lugar.
- **Isolamento** — o Postgres do projeto não conflita com outro instalado.
- **Paridade dev/prod** — a imagem testada é a que vai para produção.

1.2 O Dockerfile do ScreenFlix (multi-stage)

O arquivo `Dockerfile` usa um padrão de produção: **duas etapas**. A etapa `builder` instala dependências pesadas de compilação; a etapa `runtime` recebe só o resultado — imagem final enxuta e segura.

```
# — Stage 1: builder —————
FROM python:3.13-slim AS builder
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libpq-dev && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY pyproject.toml uv.lock ./          # copiado ANTES do código → cache de deps
RUN pip install uv && uv sync --frozen --no-dev --compile-bytecode --no-install-project
COPY src/ src/

# — Stage 2: runtime —————
FROM python:3.13-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 curl && rm -rf /var/lib/apt/lists/*
RUN groupadd -r screenflix && useradd -r -g screenflix -d /app screenflix
WORKDIR /app
COPY --from=builder /app/.venv .venv/    # só o venv pronto vem do builder
COPY --from=builder /app/src src/
COPY schemas.json .
ENV PATH="/app/.venv/bin:$PATH"
ENV PYTHONPATH="/app/src"
USER screenflix                          # non-root: boa prática de segurança
HEALTHCHECK --interval=30s --timeout=3s \
    CMD curl -f http://localhost:8000/healthz || exit 1
EXPOSE 8000
CMD ["uvicorn", "screenflix.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Técnica	Por que importa
Multi-stage	Ferramentas de build ficam fora da imagem final → menor e mais segura.
Ordem dos <code>COPY</code>	<code>pyproject.toml</code> / <code>uv.lock</code> antes do código: alterar código não reinstala dependências (cache de camadas).
<code>--no-install-recommends</code> + limpar apt	Reduz tamanho e superfície de ataque.
Usuário <code>screenflix</code> (non-root)	O container não roda como root.
<code>HEALTHCHECK</code> em <code>/healthz</code>	O orquestrador sabe se o container está saudável. É um contrato de infra.

DEMONSTRAÇÃO

```
docker build -t screenflix:latest .
docker images screenflix      # observe o tamanho da imagem final
docker history screenflix     # veja as camadas geradas
```

1.3 Orquestrando serviços com docker-compose

O `docker-compose.yml` descreve dois serviços ativos: o banco (`db`) e a aplicação (`api`). O `api` é construído a partir do `Dockerfile` e sobe só depois do banco estar saudável.

```
services:
  db:
    image: postgres:18-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql # bootstrap do schema
    ports:
      - "5532:5432" # host:container – 5532 evita conflito com Postgres local
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 3

  api:
    build: # constrói do Dockerfile (não usa imagem pronta)
      context: .
      dockerfile: Dockerfile
      target: runtime
    ports:
      - "8000:8000"
    env_file: .env
    volumes:
      - ./src:/app/src # bind-mount → hot-reload em dev
      - ./schemas.json:/app/schemas.json
    depends_on:
      db:
        condition: service_healthy # só sobe após o banco ficar saudável
    command: uvicorn screenflix.main:app --host 0.0.0.0 --port 8000 --reload --reload-dir src
```

Conceitos: `image` (imagem pronta, no `db`) vs `build` (constrói do Dockerfile, no `api`); variáveis `${...}` e `env_file` vindas do `.env`; o volume que monta `scripts/init.sql`, executado pelo Postgres na primeira inicialização (cria as tabelas `media` e `episode`); e o `depends_on ... service_healthy`, que faz o `api` esperar o healthcheck do banco antes de subir.

Crie o arquivo `.env` na raiz (não versionado)

```
POSTGRES_USER=screenflix
POSTGRES_PASSWORD=screenflix
POSTGRES_DB=screenflix

SCREENFLIX_APP_NAME=ScreenFlix
SCREENFLIX_APP_DESCRIPTION="Catálogo de filmes e séries"
SCREENFLIX_DATABASE_URL=postgresql+asyncpg://screenflix:screenflix@db:5432/screenflix
SCREENFLIX_OPENAI_API_KEY=sk-... # placeholder no workshop
SCREENFLIX_OMDB_API_KEY=... # placeholder no workshop
```

DEMONSTRAÇÃO — SUBIR O STACK

```
docker compose up --build # sobe db + api
docker compose logs -f db # veja o init.sql rodando
docker compose exec db psql -U screenflix -d screenflix -c "\dt"
curl http://localhost:8000/healthz # a api já responde
```

MÃOS À OBRA

O serviço `db` hoje **não persiste dados** — o volume nomeado `db-data` está comentado no topo do arquivo. Habilite a persistência: descomente `volumes: db-data:` e adicione `- db-data:/var/lib/postgresql/data` ao serviço `db`. Suba, registre uma mídia, derrube com `docker compose down` (sem `-v`) e confirme que os dados sobrevivem ao reiniciar. Note também que o serviço `api` usa `volumes: ./src:/app/src` + `--reload` para hot-reload — o mesmo Dockerfile serve dev e prod.

1.4 Comandos essenciais (cheat sheet)

Objetivo	Comando
Build da imagem	<code>docker build -t screenflix:latest .</code>
Subir stack (background)	<code>docker compose up -d</code>
Subir com rebuild	<code>docker compose up --build</code>
Ver serviços	<code>docker compose ps</code>
Logs (seguir)	<code>docker compose logs -f api</code>
Shell no container	<code>docker compose exec api bash</code>
Parar tudo	<code>docker compose down</code>
Parar e apagar volumes	<code>docker compose down -v</code>

1.5 Makefile — a interface única do projeto

O projeto **já traz** um `Makefile` completo. Ninguém decora `uv run uvicorn screenflix.main:app --reload --reload-dir src --host 0.0.0.0 --port 8000` — com o Makefile isso vira `make dev`. Ele é a **interface única** para dev, CI e IA (Dia 3).

O topo do arquivo traz configurações que valem discussão:

```
SHELL := bash
.SHELLFLAGS := -eu -o pipefail -c # falha cedo: erro em qualquer etapa aborta o alvo
COMPOSE := docker compose # variável → um só lugar para trocar
.DEFAULT_GOAL := help # `make` sem argumento mostra a ajuda

help: ## Mostra esta ajuda (varre os comentários ## do próprio arquivo)
    @grep -E '^[a-zA-Z0-9_]+:.*?## .*$$' $(MAKEFILE_LIST) | sort \
        | awk 'BEGIN{FS=":.*?## "};{printf " \033[36m%-18s\033[0m %s\n",$$1,$$2}'
```

Os alvos são organizados por seção:

Seção	Alvos
Setup	<code>install</code> (uv sync) · <code>lock</code>
Dev	<code>dev</code> (API com reload) · <code>shell</code> (REPL do projeto)
Qualidade	<code>lint</code> · <code>format</code> · <code>format-check</code> · <code>typecheck</code> · <code>test</code> · <code>test-unit</code> · <code>test-integration</code> · <code>check</code>
Docker	<code>up</code> · <code>up-build</code> · <code>down</code> · <code>build</code> · <code>logs</code> · <code>ps</code> · <code>restart</code>
Frontend	<code>front-install</code> · <code>front-dev</code> · <code>front-build</code> · <code>front-preview</code>
Limpeza	<code>clean</code> (remove caches de pytest/mypy/ruff)

Destaques da seção Qualidade: `check`: `lint` `typecheck` `test` é o **CI local** (formatação fica no alvo à parte `format` / `format-check`); e os testes são fatiados pelos **markers** do `pyproject.toml` — `test-unit` (`-m "not integration"`) e `test-integration` (`-m integration`).

MÃOS À OBRA — ESTENDER O MAKEFILE

O Makefile não tem um atalho para o `psql`. Adicione, na seção Docker, um alvo `db-shell` e confirme que ele aparece **ordenado** em `make help`:

```
db-shell: ## Abre o psql no banco
    $(COMPOSE) exec db psql -U ${POSTGRES_USER} -d ${POSTGRES_DB}
```

ARMADILHA DO MAKE

variáveis de shell precisam de `$$` (ex.: `${POSTGRES_USER}`) para o Make não interpolá-las — quem resolve é o shell, em runtime.

Resumo do Dia 1

- Container = ambiente reproduzível; o Dockerfile **multi-stage** entrega imagem enxuta, non-root e com healthcheck.
- **docker-compose** orquestra app (`api`) + banco (`db`); `init.sql` faz o bootstrap do schema.
- O **Makefile** é a interface única (dev, qualidade, docker, frontend); `make check` é o CI local.

Programação Pragmática

Objetivo: abrir a "caixa" empacotada no Dia 1 e entender como o código se organiza — ambientes modulares e Clean Architecture — sem dogmatismo.

2.1 A tese: arquitetura serve à mudança

Arquitetura não é enfeite; existe para tornar a mudança **barata e segura**. "Pragmática" = aplicar princípios na medida certa. O ScreenFlix se descreve como um **monólito modular em camadas**: um único deploy (simples de operar), mas internamente bem organizado (fácil de evoluir). É o ponto ideal antes de pensar em microsserviços.

2.2 Ambientes modulares

Um ambiente é "modular" quando a aplicação **não sabe** onde roda — ela apenas lê configuração de fora. Assim, a mesma imagem roda em dev, teste e produção.

Backend — `pydantic-settings`

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix="SCREENFLIX_",  # toda config vem de SCREENFLIX_*
        env_file=".env",
        extra="ignore",
    )
    app_name: str  # obrigatório (sem default)
    database_url: str  # obrigatório
    database_echo: bool = False  # com default e tipado
    openai_api_key: str  # obrigatório
    omdb_api_key: str  # obrigatório

    def get_settings() -> Settings:  # singleton cacheado
        global _settings
        if _settings is None:
            _settings = Settings()
        return _settings
```

- **Tipagem forte** — Pydantic valida e converte na inicialização. Config inválida falha na hora (*fail fast*), não vira bug em runtime.
- **Obrigatório vs default** — campos sem default tornam impossível subir mal configurado.
- **Segurança** — segredos vêm do ambiente; o `.env` está no `.gitignore`.

Frontend — variável de build do Vite

```
export const API_BASE = import.meta.env.VITE_API_BASE || "";
```

Mesmo princípio: a URL da API vem de `VITE_API_BASE`. Detalhe elegante — se estiver vazia, o app entra em **"Modo demonstração"** (offline) sem quebrar a interface.

2.3 O mapa das camadas

```
src/screenflix/
├─ main.py           # endpoint FastAPI (fino: middleware, router, /healthz)
├─ core/             # preocupações transversais (compartilhadas)
│   ├── settings.py  # configuração
│   ├── database.py  # engine/session async, Base, mixins
│   ├── repository.py # BaseRepository[T] genérico
│   ├── app/         # dependencies (DI) + middlewares
│   └─ logging/      # structlog (config, trace_id, logger)
└─ modules/          # módulos de negócio (bounded contexts)
    └─ catalog/
        ├── domain/      # camada 1 – entidades
        ├── application/  # camada 2 – services, use cases, schemas
        ├── infrastructure/ # camada 3 – repositórios (persistência)
        ├── adapters/    # integrações externas (OMDb, OpenAI)
        └─ presentation/ # camada 4 – API HTTP (FastAPI)
```

O que é framework/infra compartilhada fica em `core/`; regra de negócio fica em `modules/<contexto>/`. Para um novo contexto (ex.: `users`), replica-se a mesma estrutura de camadas — o padrão é repetível e previsível.

2.4 Clean Architecture, camada por camada

Domain — entidades

`Media` e `Episode` são o coração do negócio; `MediaType` é `movie` / `series`. Usam mixins (`IDMixin`, `TimestampMixin`).

TRADE-OFF PRAGMÁTICO

As entidades **são modelos SQLAlchemy** (herdam `Base`). Na Clean Architecture "pura", o domínio seria independente de framework. Aqui trocou-se portabilidade por menos boilerplate — uma escolha consciente que vale debater.

Application — services, use cases, schemas

Service = operação simples e reutilizável. **Use case** = orquestração de um objetivo de negócio. Exemplo de service:

```
class MediaService:
    def __init__(self, session: AsyncSession):
        self.repository = RepositoryFactor(session)
    async def top_five(self, media_type: str) -> list[Media]:
        return await self.repository.media.list_all(
            limit=5, media_type=media_type, order_by="rating", desc=True)
```

O use case `RegisterDataWorkflow.execute()` orquestra um fluxo rico: busca na OMDb → normaliza via OpenAI (com `asyncio.Semaphore(3)` limitando concorrência) → persiste `Media` e `Episode` s em paralelo → `commit()` ou `rollback()` em erro (transação única). Os **schemas** Pydantic são a fronteira do contrato: nunca se expõe a entidade direto.

Infrastructure — repositórios

O padrão **Repository** isola o acesso a dados. A base genérica evita repetição:

```
class BaseRepository(Generic[T]):
    def __init__(self, session: AsyncSession, model: Type[T]):
        self.session = session; self.model = model
    async def get_by_id(self, id: int) -> T | None: ...
    async def list_all(self, skip=0, limit=50, order_by=None, desc=False, **filters): ...
    async def count(self, **filters) -> int: ...

class MediaRepository(BaseRepository[Media]): pass # herda tudo
```

Presentation — API + Injeção de Dependência

O endpoint é **fino**: valida, delega, devolve. A DI encadeia sessão → service → endpoint:

```
async def get_media_service(session: AsyncSession = Depends(get_db_session)) -> MediaService:
    return MediaService(session)

@router.get("/movies/top5", response_model=list[MediaBaseSchema])
async def list_top_five_movies(service: MediaService = Depends(get_media_service)):
    return await service.top_five(media_type="movie")
```

O `response_model=` garante que a saída sempre obedece ao contrato.

2.5 A regra de dependência (o coração de tudo)

As dependências apontam **para dentro**:

```
presentation → application → domain
|
infrastructure ——— (depende de domain + core)
adapters —————→ core.settings / domain
```

- `domain` **não** importa `presentation`.
- `presentation` **não** contém regra de negócio (só transporte HTTP).
- Nunca consulte o banco direto de um endpoint — sempre `service` → `repository`.

MÃOS À OBRA

Adicione `GET /api/v1/media/count` respeitando as camadas: (1) método `count()` no `MediaService` usando `self.repository.media.count()`; (2) handler fino em `enpoints/media.py` delegando ao service; (3) um DTO Pydantic de resposta; (4) testes (happy path + erro). Valide com `make check`. **Regra de ouro:** nenhuma query no endpoint.

2.6 Onde o ScreenFlix escolheu pragmatismo

- **Entidades = modelos ORM** — simplicidade sobre portabilidade.
- **register fire-and-forget** — o endpoint dispara `asyncio.create_task` e responde na hora; sem fila durável, o trabalho se perde se o processo cair (dívida consciente).
- **Top 5 no frontend** — apesar de existirem endpoints `/top5`, a Home calcula no cliente.

Resumo do Dia 2

- Configuração **injetada de fora** (pydantic-settings, Vite) → ambientes modulares.
- Quatro camadas com responsabilidades claras + **regra de dependência** = mudança segura.
- Clean Architecture é um **espectro**: o valor está em saber, conscientemente, onde relaxar.

DIA 3 · 1H30

IA no Desenvolvimento

Objetivo: usar o **Claude Code** de forma disciplinada — fazendo a IA **respeitar** a infra do Dia 1 e a arquitetura do Dia 2. (Formatos válidos para o Claude Code v2.1.x, jul/2026.)

3.1 O insight do ScreenFlix

Este repositório **já foi construído pensando em IA**. Ele tem um `AGENTS.md` (ponto de entrada), um `AI-CONFIG.json` (regras de runtime) e uma pasta `/agents` com 7 guias de domínio. Essa camada de contexto é **genérica** (Markdown + JSON). O trabalho do dia é **traduzi-la** em artefatos **nativos** do Claude Code:

Contexto genérico do repo	Artefato nativo do Claude Code
<code>AGENTS.md</code>	<code>CLAUDE.md</code> (memória de projeto)
<code>AI-CONFIG.json</code> → <code>quality_checks</code>	Skill <code>/quality-gate</code> + Hook
<code>agents/backend-development.md</code>	Subagent <code>backend-dev</code>
<code>agents/qa-guidelines.md</code>	Subagent <code>qa-reviewer</code>
<code>agents/security-check.md</code>	Subagent <code>security-check</code>
vários subagents atuando juntos	Agent Team

3.2 CLAUDE.md — a memória do projeto

É o arquivo que o Claude Code lê no início de **toda** sessão. Vive em `./CLAUDE.md` (projeto, versionado) ou `~/.claude/CLAUDE.md` (usuário).

ATENÇÃO

O Claude Code lê `CLAUDE.md`, **não** `AGENTS.md`. Como o ScreenFlix já investiu num `AGENTS.md` rico, a jogada é **importar**, não duplicar.

```
# ScreenFlix — Contexto para Claude Code
```

```
@AGENTS.md
```

```
@AI-CONFIG.json
```

```
@agents/architecture-planning.md
```

```
## Regras rápidas
```

- Arquitetura em camadas: nunca acesse o banco a partir da presentation.
- Rode os quality gates antes de concluir: ruff format, ruff check, mypy src, pytest.
- Todo componente novo exige testes (happy path, validação, erro).
- Segredos vêm do ambiente (SCREENFLIX_*). Nunca commite .env.

O `@AGENTS.md` puxa o arquivo existente para o contexto, reaproveitando o trabalho do time.

3.3 Skills — encapsulando procedimentos

Uma **Skill** é um procedimento reutilizável, carregado **sob demanda**, que vira um comando `/nome`. Vive em `.claude/skills/<nome>/SKILL.md`.

```
---
name: quality-gate
description: Roda os quality gates bloqueantes do ScreenFlix. Use antes de concluir mudanças.
invocation: both
---
```

Execute, nesta ordem, parando no primeiro que falhar:

1. ``uv run ruff format .``
2. ``uv run ruff check .``
3. ``uv run mypy src``
4. ``uv run pytest``

Se algum falhar, resuma o erro e proponha a correção mínima. Não considere a tarefa concluída enquanto os quatro não passarem.

HOOK (BÔNUS)

Para **forçar** o gate automaticamente, configure um hook em `.claude/settings.json` no evento `Stop`. Hooks tornam comportamentos automáticos — a máquina executa, não o modelo.

3.4 Subagents — especialistas por domínio

Um **Subagent** é um trabalhador **isolado**, com contexto e ferramentas próprios, ao qual o Claude principal **delega** tarefas conforme a `description`. Vive em `.claude/agents/<nome>.md`. Cada guia de `/agents/` vira um subagent:


```
---
name: qa-reviewer
description: Revisor de QA do ScreenFlix. Use após alterar código para checar cobertura de testes.
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

Você é o revisor de QA. Fonte de verdade: agents/qa-guidelines.md.
Ao revisar: identifique componentes novos; exija testes (happy path, validação, erro);
verifique thresholds ($\geq 80\%$ módulos alterados); rode ``uv run pytest``. Se faltar teste
para componente novo, marque como BLOQUEANTE e liste o que testar.

Subagent	Traduz de	Papel
<code>architect</code>	architecture-planning.md	Protege a regra de dependência
<code>backend-dev</code>	backend-development.md	Padrões FastAPI async, mypy strict
<code>database-dev</code>	database-development.md	Repos/migrations, sem query na presentation
<code>security-check</code>	security-check.md	Segredos via env, "fail closed"
<code>qa-reviewer</code>	qa-guidelines.md	Cobertura e política de testes

Skill vs Subagent: a Skill roda na conversa principal (procedimento); o Subagent roda isolado, com contexto próprio, e devolve um resultado.

3.5 Agent Teams — orquestração multi-agente

EXPERIMENTAL

Recurso desativado por padrão (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`). Apresentado como direção do futuro, não fluxo de produção estável.

Aspecto	Subagents	Agent Teams
Comunicação	Reportam ao agente principal	Teammates conversam entre si
Coordenação	Agente principal gerencia	Lista de tarefas compartilhada
Uso ideal	Tarefas focadas e rápidas	Trabalho paralelo complexo

Um **team lead** spawna teammates independentes que compartilham uma lista de tarefas e herdam `CLAUDE.md` , Skills e Subagents do projeto. Regra prática: comece simples — subagent para mudanças focadas; team só para trabalho amplo e paralelizável.

Resumo do Dia 3

- Contexto de engenharia (Dias 1 e 2) vira **contexto executável de IA**.
- `CLAUDE.md` importa o que já existe; **Skills** encapsulam procedimentos; **Subagents** são especialistas; **Agent Teams** coordenam vários.
- Os artefatos de IA **fazem cumprir** a arquitetura — o `qa-reviewer` exige testes, o `architect` protege as camadas.

Cheat sheet, glossário e referências

A.1 Comandos de bolso

Contexto	Comando
Instalar deps	<code>uv sync --dev</code>
API local (reload)	<code>uv run uvicorn screenflix.main:app --reload --reload-dir src --port 8000</code>
Stack completo	<code>docker compose up --build</code>
Quality gates	<code>uv run ruff format . && uv run ruff check . && uv run mypy src && uv run pytest</code>
psql no banco	<code>docker compose exec db psql -U screenflix -d screenflix</code>
Frontend (dev)	<code>cd frontend && npm install && npm run dev</code>

A.2 Glossário

Termo	Significado
Multi-stage build	Dockerfile com etapas separadas (build vs runtime) para imagem final enxuta.
Healthcheck	Verificação periódica que diz ao orquestrador se o container está saudável.
Monólito modular	Um único deploy, internamente dividido em módulos e camadas.
Clean Architecture	Organização em camadas com dependências apontando para dentro.
Repository	Padrão que isola o acesso a dados da lógica de negócio.
Use case	Orquestração de um objetivo de negócio (vários passos/adapters).
Injeção de dependência	Fornecer dependências prontas (ex.: sessão de banco) em vez de criá-las dentro.
CLAUDE.md	Arquivo de memória/contexto que o Claude Code lê a cada sessão.
Skill	Procedimento reutilizável carregado sob demanda; vira comando <code>/nome</code> .
Subagent	Agente isolado e especializado ao qual o Claude principal delega.
Agent Team	Vários agentes independentes coordenando via lista de tarefas (experimental).

A.3 Arquivos-chave do repositório

Tema	Arquivo
Container	Dockerfile , docker-compose.yml , scripts/init.sql
Config/tooling	pyproject.toml (uv, ruff, mypy, pytest)
Arquitetura	src/screenflix/core/ , src/screenflix/modules/catalog/
Frontend	frontend/ (Vue 3 + Pinia)
Contexto de IA	AGENTS.md , AI-CONFIG.json , agents/*.md
Roteiros do instrutor	docs/workshop/dia-1..3.md

ScreenFlix Workshop · Material didático para os participantes · 3 encontros de 1h30
Formatos do Claude Code válidos para a versão v2.1.x (jul/2026).